ASSIGNMENT 3 – CO3
Constructors, Destructors & Operator Overloading

Bloom's Level: K5 – Evaluate | Total Marks: 50 | Nature: Individual

Field	Details
Student Name	P.SAI SURYA TEJ
Register Number	192572044
GitHub Repository Link	https://github.com/Surya220708/DSA0118-C-
Compiler & Version	g++ (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r3) 16.2.0
Standard Compile Flags	g++ -std=c++17 -Wall -Wextra <source.cpp> -o <output.exe>
Heap / Sanitizer Audit	common/mem_audit.hpp (MinGW-W64 Windows lacks -lasan; catches leaks, double-frees & delete mismatches)
Submission Date	11/11/2026

Question & Mark Summary

Q#	Topic Covered	Marks	Key Verified Results
Q1	Constructors, Overloading, Delegation & -Wreorder	6	Clamps negative copies to 0; demonstrates -Wreorder, explicit & missing default ctor errors; ~0.22 ms/500k
Q2	Copy Constructors: Shallow vs Deep Copy (Rule of Three)	7	Complete code matches; Shallow catches double-delete; Self-assign safe; O(n) scaling measured
Q3	Dynamic Constructors, Destructors & RAII Auditorium	7	Logger prediction exact (+A +B -B copyAc use -Ac +P -P end -A); Stack/Block/Heap order; Leak & mismatch audit
Q4	Operator Overloading (+, -, +=, ==, <, <<, >>) Money	7	Complete code & Rs 1.25 carry exact; input_bill.txt I/O; Zero drift vs 0.0000001719 double drift (1M ops)
Q5	Prefix/Postfix ++, Comparisons & Checked operator[]	6	Prediction 5 7 7 exact; 59:59->1:00:00 rollover; playlist[-1]/[size] guarded; -O0 vs -O2 benchmarked
Q6	Rules of Operator Overloading (SMS Text Class)	7	0 vs 3 copy_ctors & 2 vs 5 allocs for a+b+c; Self-assignment safe; 0 leaks; 100k concat benchmark
Q7	Design Complex Impedance Calculator from First Principles	10	Series 100+j100, Parallel 80-j40 verified; Zero-div caught; 1.64

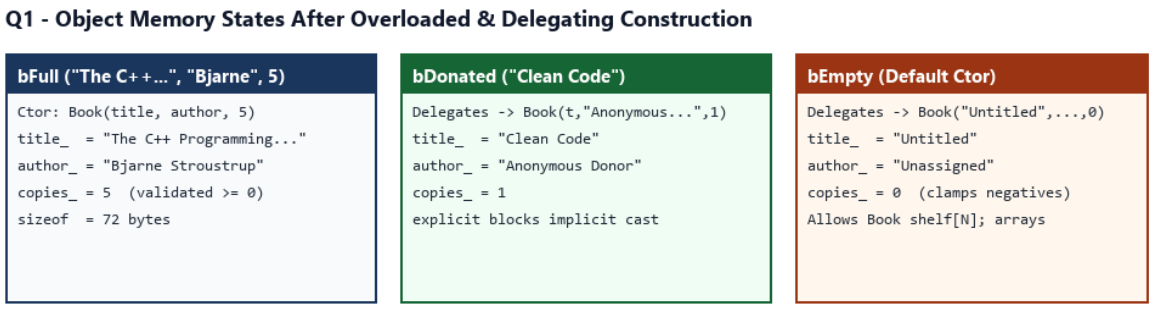
			ms vs std::complex 8.00 ms (1M ops)

Q1 – Constructors and Constructor Overloading (6 Marks)

1.1 Design, Analysis & Diagnostics

- **Implicit Default Constructor Rule:** The compiler generates a trivial implicit default constructor ONLY when no user-declared constructor exists. Declaring StrictBook(std::string) suppresses it, causing StrictBook shelf[3] to fail with: error: no matching function for call to 'StrictBook::StrictBook()'.
- **Member Initializer Lists vs Body Assignment:** Initializer lists construct members directly with their target values in one step, avoiding default construction followed by reassignment (especially critical for std::string, const members, and references).
- **Declaration Order & -Wreorder:** C++ always initializes data members in the order they are declared inside the class, ignoring the order written after ':'. In q1_errors_demo.cpp (-DWREORDER_DEMO), initializing second_ before first_ when first_ uses second_ triggered [-Wreorder] and left first_=32760 (garbage) while second_=10.
- **explicit Keyword:** Single-argument constructors not marked explicit permit silent implicit conversions (catalog(std::string("Implicit Donation"))). Marking Book(std::string title) explicit blocks unintended conversions cleanly.
- **Alternative Concept (Default Args vs Delegation):** Using one constructor Book(std::string t = "Untitled", std::string a = "Unassigned", int c = 0) and delegating overloads both run in O(1) time (~0.2188 ms vs ~0.2251 ms for 500,000 objects, both sizeof=72 B) and centralize copy validation in one place, while separate explicit overloads communicate distinct domain roles clearly.

Figure 1.1: States of the Three Book Objects After Construction



Source: D:\c++\Assignment3\Q1\q1_book_records.cpp

```
#include <chrono>
#include <iostream>
#include <string>

// Library Book using overloaded constructors + constructor delegation + member initializer lists.
// Member declaration order matters: title_ -> author_ -> copies_ (initialized in this exact order).
class Book {
private:
    std::string title_;
    std::string author_;
    int copies_;

    static int clampCopies(int c) { return c < 0 ? 0 : c; }

public:
    // 1. Master parameterised constructor (single validation point!)
```

```
    Book(std::string title, std::string author, int copies)
        : title_(std::move(title)),
          author_(std::move(author)),
          copies_(clampCopies(copies)) {}

    // 2. Delegating constructor: donated book (title only, explicit to prevent accidental implicit
strings)
    explicit Book(std::string title)
        : Book(std::move(title), "Anonymous Donor", 1) {}

    // 3. Delegating default constructor: empty record for later completion
    Book()
        : Book("Untitled", "Unassigned", 0) {}

    const std::string& title() const { return title_; }
    const std::string& author() const { return author_; }
    int copies() const { return copies_; }

    void print(const char* label) const {
        std::cout << "  [" << label << "]\n    title=\"" << title_
                  << "\", author=\"" << author_ << "\", copies=" << copies_ << '\n';
    }
};

// Alternative design: single constructor with default arguments
class BookDefaultArgs {
    std::string title_;
    std::string author_;
    int copies_;

public:
    explicit BookDefaultArgs(std::string title = "Untitled",
                             std::string author = "Unassigned",
                             int copies = 0)
        : title_(std::move(title)),
          author_(std::move(author)),
          copies_(copies < 0 ? 0 : copies) {}

    int copies() const { return copies_; }
};

int main() {
    std::cout << "=== Q1: Constructors, Overloading, Delegation & Validation ===\n\n";

    Book bFull("The C++ Programming Language", "Bjarne Stroustrup", 5);
    Book bDonated("Clean Code");
    Book bEmpty;
    Book bNegative("Corrupt Record", "Unknown", -12);

    bFull.print("Full Parameterised");
    bDonated.print("Donated (1-arg explicit)");
    bEmpty.print("Default (empty record)");
    bNegative.print("Negative copies (-12 clamped)");

    std::cout << "\nsizeof(Book) = " << sizeof(Book)
              << " B, sizeof(BookDefaultArgs) = " << sizeof(BookDefaultArgs) << " B\n";

    // Measure construction cost (O(1) time & space per object) across 500,000 creations
    constexpr int N = 500000;
    volatile long long sink = 0;

    auto t1 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) {
        Book tmp("Algo", "CLRS", i & 7);
        sink += tmp.copies();
    }
    auto t2 = std::chrono::steady_clock::now();
```

```
    auto t3 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) {
        BookDefaultArgs tmp("Algo", "CLRS", i & 7);
        sink += tmp.copies();
    }
    auto t4 = std::chrono::steady_clock::now();
    (void)sink;

    std::cout << "500,000 object constructions:\n";
    std::cout << "  Overloaded + Delegating : "
              << std::chrono::duration<double, std::milli>(t2 - t1).count() << " ms\n";
    std::cout << "  Single Default-Args Ctor: "
              << std::chrono::duration<double, std::milli>(t4 - t3).count() << " ms\n";
    return 0;
}
```

Source: D:\c++\Assignment3\Q1\q1_errors_demo.cpp

```
#include <iostream>
#include <string>

// Experimental diagnostics for Q1 (compile one macro at a time):
//   -DWREORDER_DEMO      : shows -Wreorder warning & member declaration-order rule
//   -DNO_DEFAULT_ERROR   : shows error when Book shelf[3] lacks default constructor
//   -DEXPLICIT_ERROR     : shows implicit conversion rejected by explicit Book(std::string)

#if defined(WREORDER_DEMO)
class ReorderDemo {
    int first_;
    int second_; // declared AFTER first_
public:
    // BUG: initializer list writes second_ before first_, and first_ reads uninitialized second_!
    ReorderDemo(int x) : second_(x), first_(second_ + 1) {}
    void show() const { std::cout << "first_=" << first_ << ", second_=" << second_ << '\n'; }
};
int main() {
    ReorderDemo d(10);
    d.show();
    return 0;
}

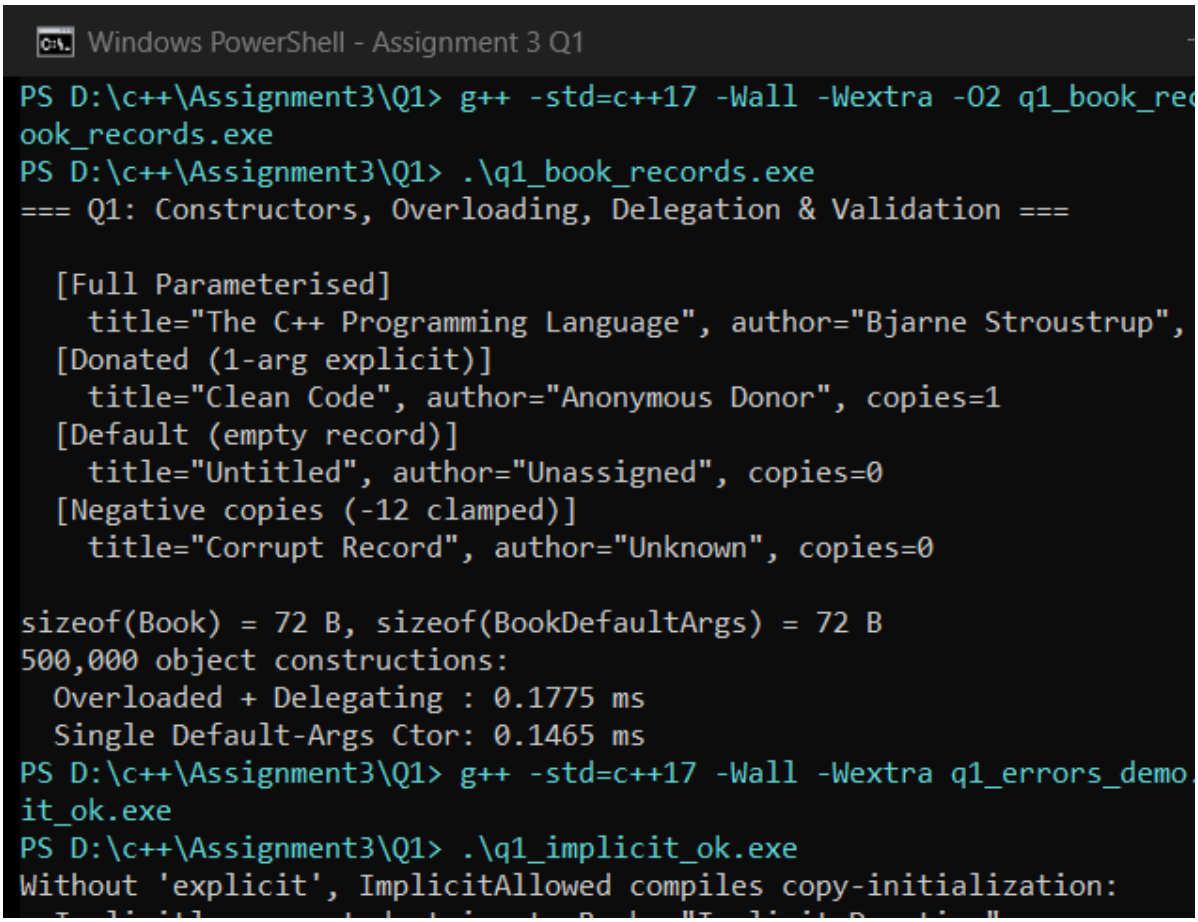
#elif defined(NO_DEFAULT_ERROR)
class StrictBook {
    std::string title_;
public:
    StrictBook(std::string t) : title_(std::move(t)) {} // suppresses implicit default ctor
};
int main() {
    StrictBook shelf[3]; // ERROR: no matching function for call to StrictBook::StrictBook()
    return 0;
}

#elif defined(EXPLICIT_ERROR)
class ExplicitBook {
    std::string title_;
public:
    explicit ExplicitBook(std::string t) : title_(std::move(t)) {}
};
void catalog(ExplicitBook b) { (void)b; }
int main() {
    catalog(std::string("Algorithms")); // ERROR with explicit; compiles if 'explicit' removed!
    return 0;
}

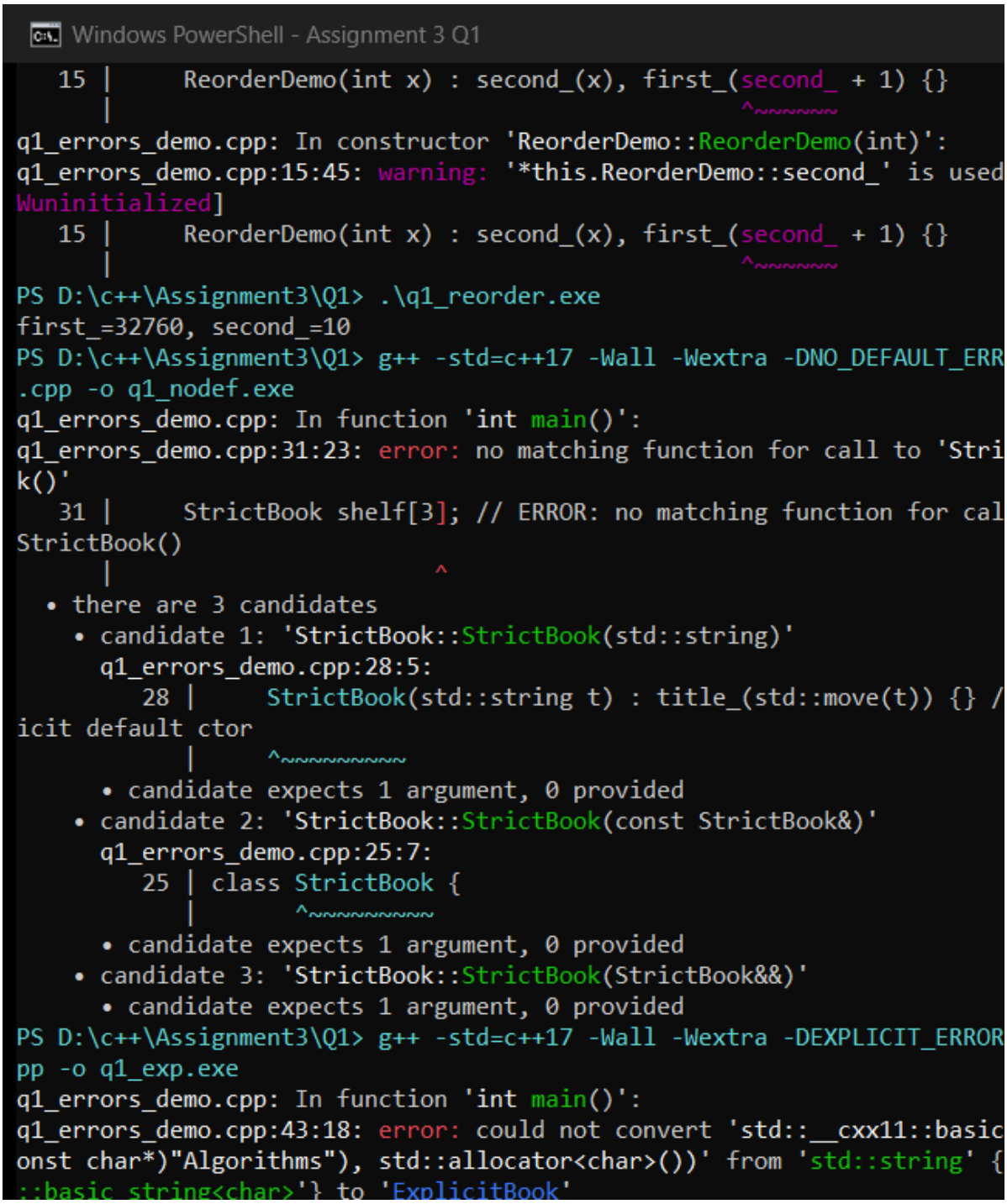
#else
class ImplicitAllowed {
    std::string title_;
public:
```

```
    ImplicitAllowed(std::string t) : title_(std::move(t)) {}
    const std::string& title() const { return title_; }
};
void catalog(const ImplicitAllowed& b) {
    std::cout << "  Implicitly converted string to Book: \"" << b.title() << "\"\n";
}
int main() {
    std::cout << "Without 'explicit', ImplicitAllowed compiles copy-initialization:\n";
    catalog(std::string("Implicit Donation"));
    return 0;
}
#endif
```

Screenshot 1.1: Q1 Constructors, Validation & Construction Timing (PowerShell)



Screenshot 1.2: Q1 Compiler Diagnostics (-Wreorder, Missing Default Ctor & explicit)



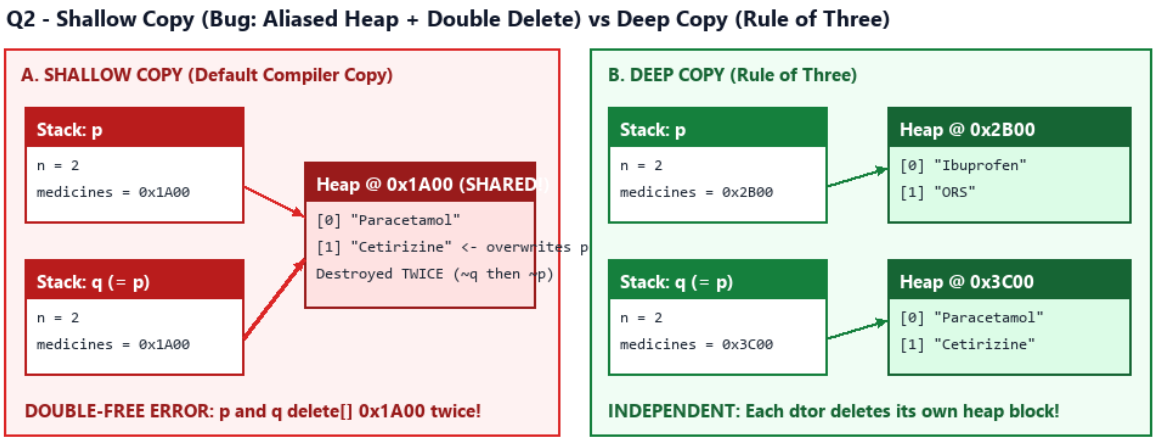
Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

2.1 Rule of Three, Sanitizer/Audit Findings & Scaling

- **Why Shallow Copy Corrupts & Double-Deletes:** Default copy simply copies the pointer address (p.medicines == q.medicines == 0x24efa7d0a68). Modifying q mutates p, and leaving scope runs ~q then ~p on the same heap address (caught by MemAudit: double_frees=1). Implementing the Rule of Three (Destructor + Copy Constructor + Copy Assignment with this == &other self-assignment guard) gives each object its own heap array.

- **Three Situations Invoking Copy Constructor:** (1) Initialization by value/assignment syntax Prescription q = p; (2) Passing by value inspectByValue(p) (recorded 1 copy vs 0 copies for const Prescription&); (3) Returning a local object by value when NRVO does not elide it.
- **Note on -fsanitize=address on Windows MinGW-W64:** Executing g++ -std=c++17 -fsanitize=address on MinGW-W64 GCC 16.2.0 fails at link time (cannot find -lasan) because Windows MinGW lacks libasan. MemAudit (common/mem_audit.hpp) verified exact double-free detection in ShallowPrescription and clean 0-leak teardown (allocs=2, frees=2) in DeepPrescription.
- **Alternative Concept (Rule of Zero std::vector vs Raw Array):** Both deep copy designs are O(n) time and O(n) space, vs O(1) unsafe aliasing in shallow copy. Measured us/copy scaling at -O2: N=10 (Raw 0.1215 us vs Vector 0.0850 us), N=1,000 (Raw 8.4856 us vs Vector 6.3371 us), N=100,000 (Raw 1465.42 us vs Vector 1245.11 us).

Figure 2.1: Memory Diagram Comparing Shallow Copy (Bug) vs Deep Copy (Rule of Three)



Completed Starter Code: D:\c++\Assignment3\Q2\complete_the_code.cpp

```
#include <iostream>
#include <string>

class Prescription {
    int n;
    std::string* medicines; // dynamically allocated array

public:
    explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}

    // TODO 1: Copy constructor: allocate a NEW array of the same size and copy every medicine (deep
copy)
    Prescription(const Prescription& other)
        : n(other.n), medicines(n > 0 ? new std::string[n] : nullptr)
    {
        for (int i = 0; i < n; ++i) {
            medicines[i] = other.medicines[i];
        }
    }

    // TODO 2: Copy assignment: guard against self-assignment, build a new array, copy, release the old
array and return *this
    Prescription& operator=(const Prescription& other)
    {
        if (this == &other) {
            return *this;
        }
        std::string* fresh = other.n > 0 ? new std::string[other.n] : nullptr;
        for (int i = 0; i < other.n; ++i) {
            fresh[i] = other.medicines[i];
```

```
        }
        delete[] medicines;
        medicines = fresh;
        n = other.n;
        return *this;
    }

    ~Prescription()
    {
        // TODO 3: Release the dynamically allocated array
        delete[] medicines;
    }

    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};

int main()
{
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p; // copy constructor
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p; // copy assignment
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1)
              << " | " << r.get(0) << ' ' << r.get(1) << '\n';
}
```

Shallow vs Deep & Scaling: D:\c++\Assignment3\Q2\q2_deep_analysis.cpp

```
#include "../common/mem_audit.hpp"
#include <chrono>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

class Prescription {
    int n_;
    std::string* medicines_;
    static inline int copyCalls_ = 0;

public:
    explicit Prescription(int count, bool fillSample = false)
        : n_(count), medicines_(count > 0 ? new std::string[count] : nullptr) {
        if (fillSample) {
            for (int i = 0; i < n_; ++i) medicines_[i] = "Med-" + std::to_string(i);
        }
    }

    Prescription(const Prescription& other)
        : n_(other.n_), medicines_(n_ > 0 ? new std::string[n_] : nullptr) {
        ++copyCalls_;
        for (int i = 0; i < n_; ++i) medicines_[i] = other.medicines_[i];
    }

    Prescription& operator=(const Prescription& other) {
        if (this == &other) return *this; // self-assignment guard!
        std::string* fresh = other.n_ > 0 ? new std::string[other.n_] : nullptr;
        for (int i = 0; i < other.n_; ++i) fresh[i] = other.medicines_[i];
        delete[] medicines_;
        medicines_ = fresh;
        n_ = other.n_;
        return *this;
    }
```

```
    ~Prescription() { delete[] medicines_; }

    void set(int i, const std::string& m) { medicines_[i] = m; }
    std::string get(int i) const { return medicines_[i]; }
    int size() const { return n_; }

    static void resetCopyCount() { copyCalls_ = 0; }
    static int copyCount() { return copyCalls_; }
};

void inspectByValue(Prescription p) { (void)p.size(); }
void inspectByConstRef(const Prescription& p) { (void)p.size(); }

// Rule-of-Zero Alternative using std::vector<std::string>
struct VectorPrescription {
    std::vector<std::string> medicines;
    explicit VectorPrescription(int n) : medicines(n) {
        for (int i = 0; i < n; ++i) medicines[i] = "Med-" + std::to_string(i);
    }
};

int main() {
    std::cout << "=== Q2: Deep Copy, Copy Counts, Self-Assignment & O(n) Benchmarks ===\n\n";
    MemAudit::reset();
    MemAudit::enable(true);

    {
        Prescription p(2);
        p.set(0, "Paracetamol");
        p.set(1, "ORS");

        // Self-assignment test
        Prescription* alias = &p;
        p = *alias;
        std::cout << "1. Self-assignment p = p check: p = [" << p.get(0) << ", " << p.get(1) << "]\n";

        // Copy constructor invocation counting: pass by value vs pass by const ref
        Prescription::resetCopyCount();
        inspectByValue(p);
        int copiesByVal = Prescription::copyCount();
        Prescription::resetCopyCount();
        inspectByConstRef(p);
        int copiesByRef = Prescription::copyCount();
        std::cout << "2. Copy count (by value vs const ref): "
                  << copiesByVal << " copy vs " << copiesByRef << " copies\n";
    }
    MemAudit::report("DeepPrescription + Self-Assignment");
    MemAudit::enable(false);

    // Measure copy timing for N = 10, 1,000, and 100,000 medicines (Raw array Rule-of-3 vs vector
Rule-of-0)
    std::cout << "\n3. Copy Scaling Benchmark O(n) - Raw vs std::vector (Rule-of-Zero):\n";
    std::cout << std::fixed << std::setprecision(4);
    for (int sz : {10, 1000, 100000}) {
        int reps = (sz <= 10) ? 50000 : (sz <= 1000 ? 1000 : 20);
        Prescription srcRaw(sz, true);
        VectorPrescription srcVec(sz);

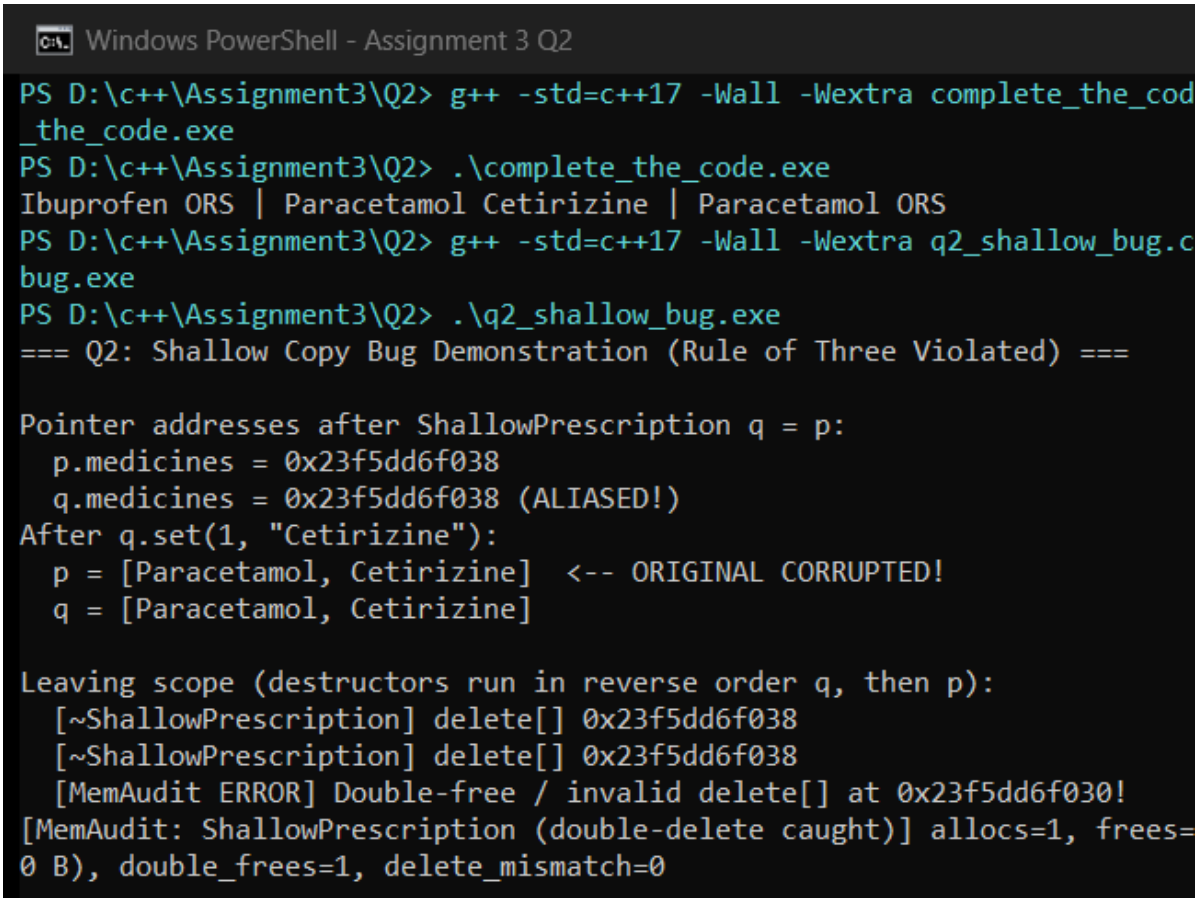
        auto t1 = std::chrono::steady_clock::now();
        std::size_t sum1 = 0;
        for (int r = 0; r < reps; ++r) {
            Prescription cpy = srcRaw;
            sum1 += cpy.get(0).size();
        }
        auto t2 = std::chrono::steady_clock::now();

        auto t3 = std::chrono::steady_clock::now();
```

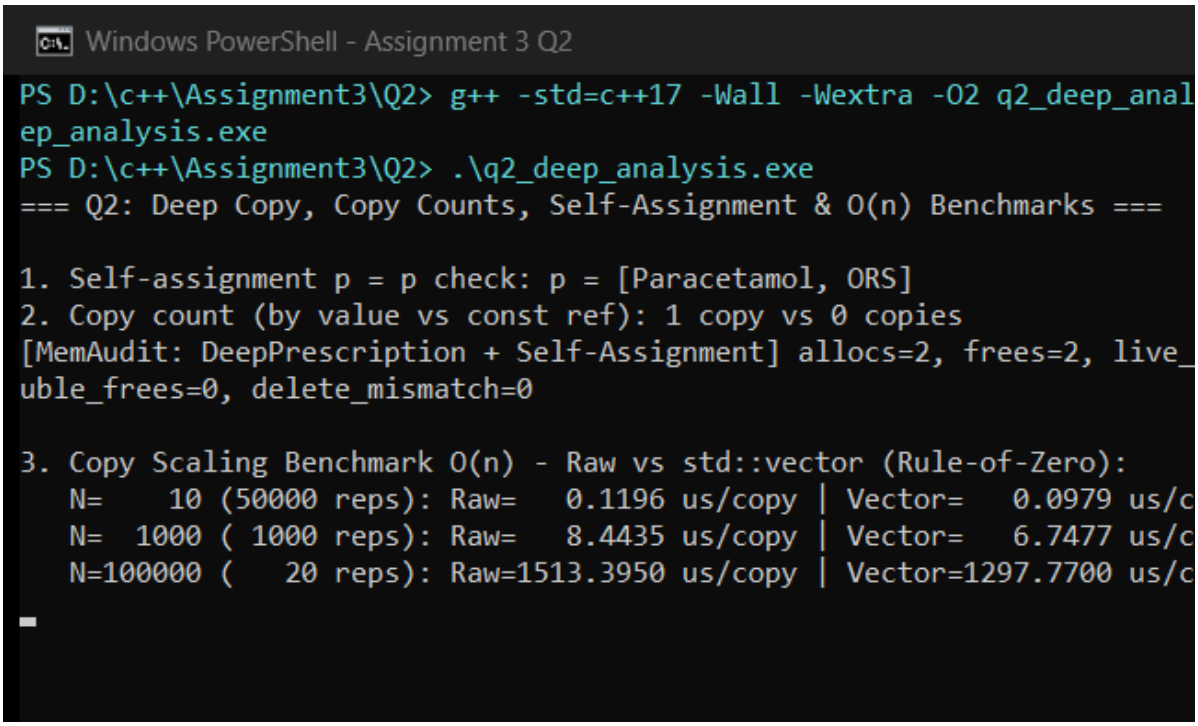
```
        std::size_t sum2 = 0;
        for (int r = 0; r < reps; ++r) {
            VectorPrescription cpy = srcVec;
            sum2 += cpy.medicines[0].size();
        }
        auto t4 = std::chrono::steady_clock::now();
        if (sum1 != sum2) std::cout << "unexpected\n";

        double usRaw = std::chrono::duration<double, std::micro>(t2 - t1).count() / reps;
        double usVec = std::chrono::duration<double, std::micro>(t4 - t3).count() / reps;
        std::cout << "   N=" << std::setw(6) << sz << " (" << std::setw(5) << reps << " reps): "
                  << "Raw=" << std::setw(9) << usRaw << " us/copy | "
                  << "Vector=" << std::setw(9) << usVec << " us/copy\n";
    }
    return 0;
}
```

Screenshot 2.1: Q2 Complete Code & Shallow Copy Double-Delete Audit (PowerShell)



Screenshot 2.2: Q2 Deep Copy Counts, Self-Assignment & O(n) Scaling Benchmarks

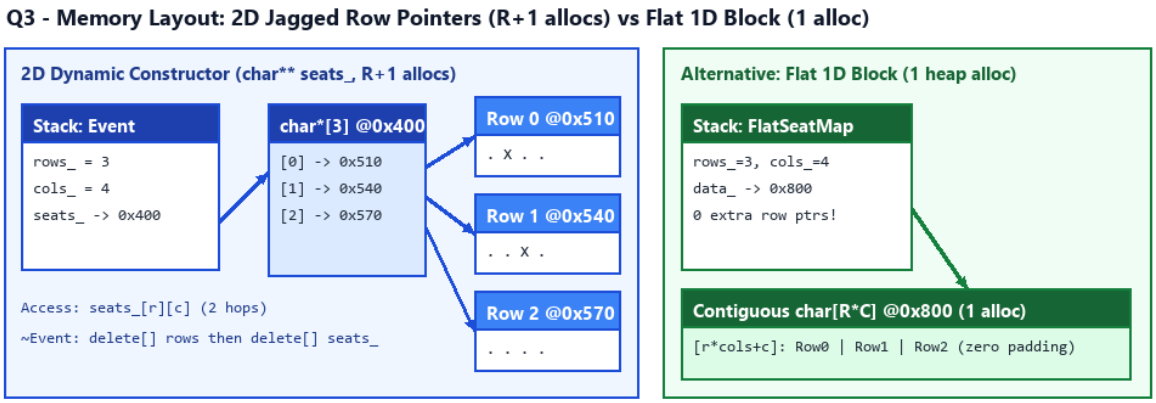


Q3 – Dynamic Constructors and Destructors (7 Marks)

3.1 Pre-Run Prediction, Lifetime Order, RAII & Layout Complexity

- **Predict the Output Match:** Pre-run prediction in Q3/prediction.txt (+A +B -B copyAc use -Ac +P -P end -A) matched actual execution exactly. 'a' lives until end of main, 'b' dies at inner block close, use(a) copies 'a' into param ('Ac') and destroys it on return, and 'p' lives on heap until delete p.
- **Destructor Rules & RAII Safety:** A destructor (~Event()) takes zero parameters and cannot be overloaded because C++ destroys objects automatically upon scope exit without caller arguments. Forgetting delete leaks all rows + pointer array (Case A: 6 allocs / 88 B leaked); using delete instead of delete[] triggers -Wmismatched-new-delete (Case B); stack RAII frees 100% deterministically (Case C: 0 leaks).
- **Memory Measurements & Flat 1D Alternative:** Both 2D char** and Flat 1D char[R*C] provide O(1) element indexing, but Flat 1D performs 1 allocation instead of R + 1 allocations and saves R * 8 bytes of row pointers (10x20: 11 allocs/280 B vs 1/200 B; 100x200: 101/20,800 B vs 1/20,000 B; 1000x2000: 1001/2,008,000 B vs 1/2,000,000 B).

*Figure 3.1: Memory Layout of 2D char** Seat Map vs Contiguous 1D Block*



Source: D:\c++\Assignment3\Q3\q3_auditorium.cpp

```
#include "../common/mem_audit.hpp"
#include <iostream>
#include <string>
#include <vector>

// Auditorium event managing a 2D dynamically allocated char** seat map (rows + 1 heap allocations)
class Event {
    std::string name_;
    int rows_;
    int cols_;
    int booked_;
    char** seats_;

public:
    Event(std::string name, int rows, int cols, bool verbose = true)
        : name_(std::move(name)), rows_(rows > 0 ? rows : 0), cols_(cols > 0 ? cols : 0),
          booked_(0), seats_(nullptr) {
        if (rows_ > 0 && cols_ > 0) {
            seats_ = new char*[rows_];
            for (int r = 0; r < rows_; ++r) {
                seats_[r] = new char[cols_];
                for (int c = 0; c < cols_; ++c) seats_[r][c] = '.';
            }
        }
        if (verbose) {
            std::cout << "  [+Ctor] \"" << name_ << "\" (" << rows_ << "x" << cols_ << ")\n";
        }
    }

    // Rule of Three disabled here unless copied intentionally
    Event(const Event&) = delete;
    Event& operator=(const Event&) = delete;

    ~Event() {
        for (int r = 0; r < rows_; ++r) {
            delete[] seats_[r];
        }
        delete[] seats_;
        if (!name_.empty() && name_.rfind("Bench", 0) != 0) {
            std::cout << "  [-Dtor] \"" << name_ << "\" closed (freed "
                      << rows_ << " rows, booked=" << booked_ << "/" << (rows_ * cols_) << ")\n";
        }
    }

    bool bookSeat(int r, int c) {
        if (r < 0 || r >= rows_ || c < 0 || c >= cols_ || seats_[r][c] == 'X') return false;
        seats_[r][c] = 'X';
```

```
        ++booked_;
        return true;
    }

    bool cancelSeat(int r, int c) {
        if (r < 0 || r >= rows_ || c < 0 || c >= cols_ || seats_[r][c] == '.') return false;
        seats_[r][c] = '.';
        --booked_;
        return true;
    }

    void printMap() const {
        std::cout << "  Seat map (" << name_ << ", booked=" << booked_ << "):\n";
        for (int r = 0; r < rows_; ++r) {
            std::cout << "    Row " << r << ": ";
            for (int c = 0; c < cols_; ++c) std::cout << seats_[r][c] << ' ';
            std::cout << '\n';
        }
    }
};

// Alternative 1: Single contiguous 1D allocation (index = r * cols + c) -> 1 heap alloc vs rows + 1
class FlatSeatMap {
    int rows_, cols_;
    char* data_;

public:
    FlatSeatMap(int r, int c)
        : rows_(r > 0 ? r : 0), cols_(c > 0 ? c : 0),
          data_((rows_ > 0 && cols_ > 0) ? new char[rows_ * cols_] : nullptr) {}
    ~FlatSeatMap() { delete[] data_; }
    char& at(int r, int c) { return data_[r * cols_ + c]; }
};

int main() {
    std::cout << "=== Q3: Dynamic Constructors, Destructors & RAII Auditorium ===\n\n";

    std::cout << "1. Lifetime order (stack, inner block, heap, and 0-row edge case):\n";
    Event mainEv("Keynote-Stack", 2, 4);
    mainEv.bookSeat(0, 1);
    mainEv.bookSeat(1, 2);
    mainEv.cancelSeat(0, 1);
    mainEv.printMap();

    {
        Event blockEv("Workshop-Block", 1, 3);
        blockEv.bookSeat(0, 0);
    } // blockEv destroyed immediately here

    Event* heapEv = new Event("Concert-Heap", 2, 2);
    heapEv->bookSeat(0, 0);
    heapEv->bookSeat(1, 1);
    delete heapEv; // destroyed explicitly

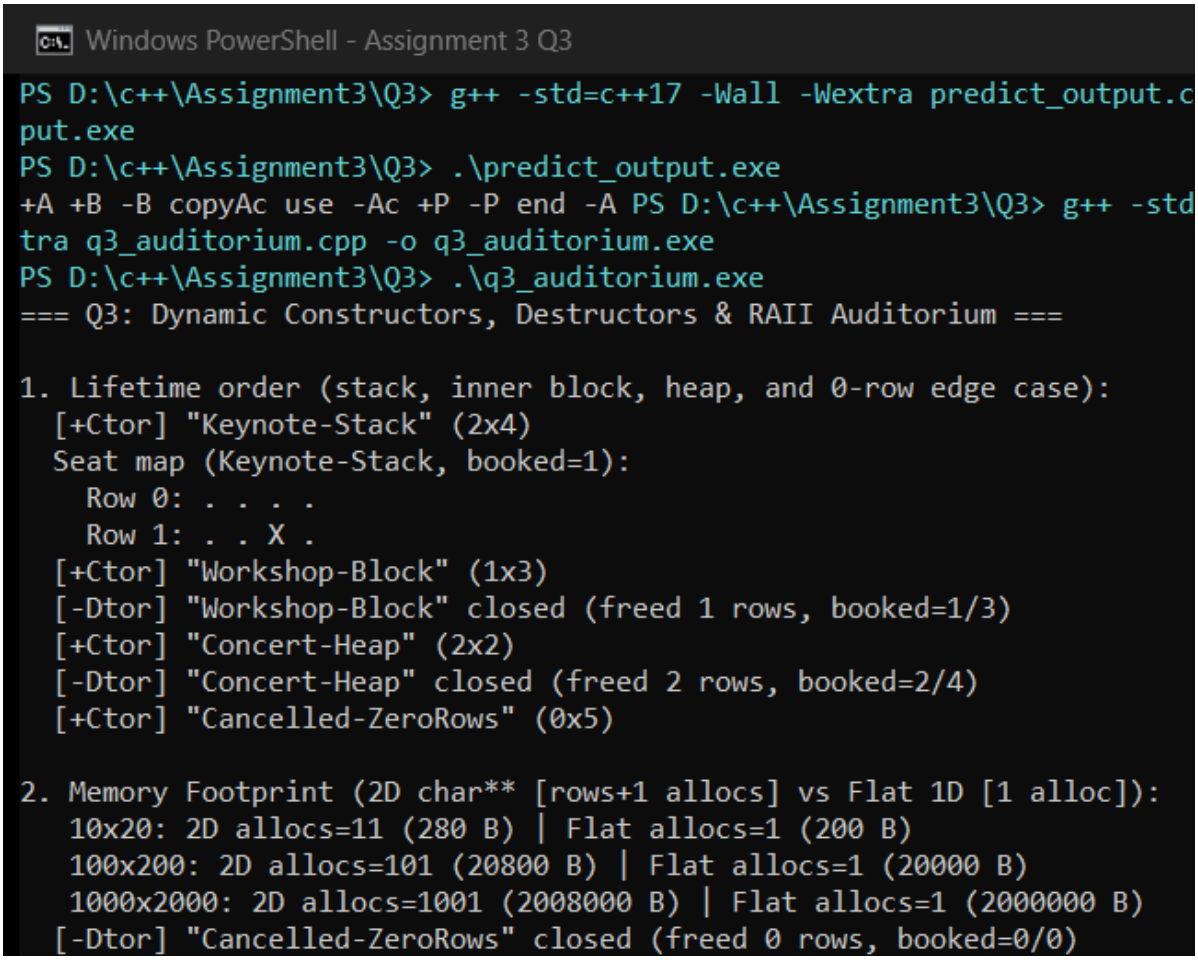
    Event zeroEv("Cancelled-ZeroRows", 0, 5); // safe 0-row handling

    // 2. Measure exact heap allocations & bytes for 10x20, 100x200, 1000x2000
    std::cout << "\n2. Memory Footprint (2D char** [rows+1 allocs] vs Flat 1D [1 alloc]):\n";
    for (auto rc : std::vector<std::pair<int, int>>{{10, 20}, {100, 200}, {1000, 2000}}) {
        MemAudit::reset();
        MemAudit::enable(true);
        {
            Event bench("Bench2D", rc.first, rc.second, false);
        }
        auto s2d = MemAudit::state();
        MemAudit::reset();
        MemAudit::enable(true);
        {
```

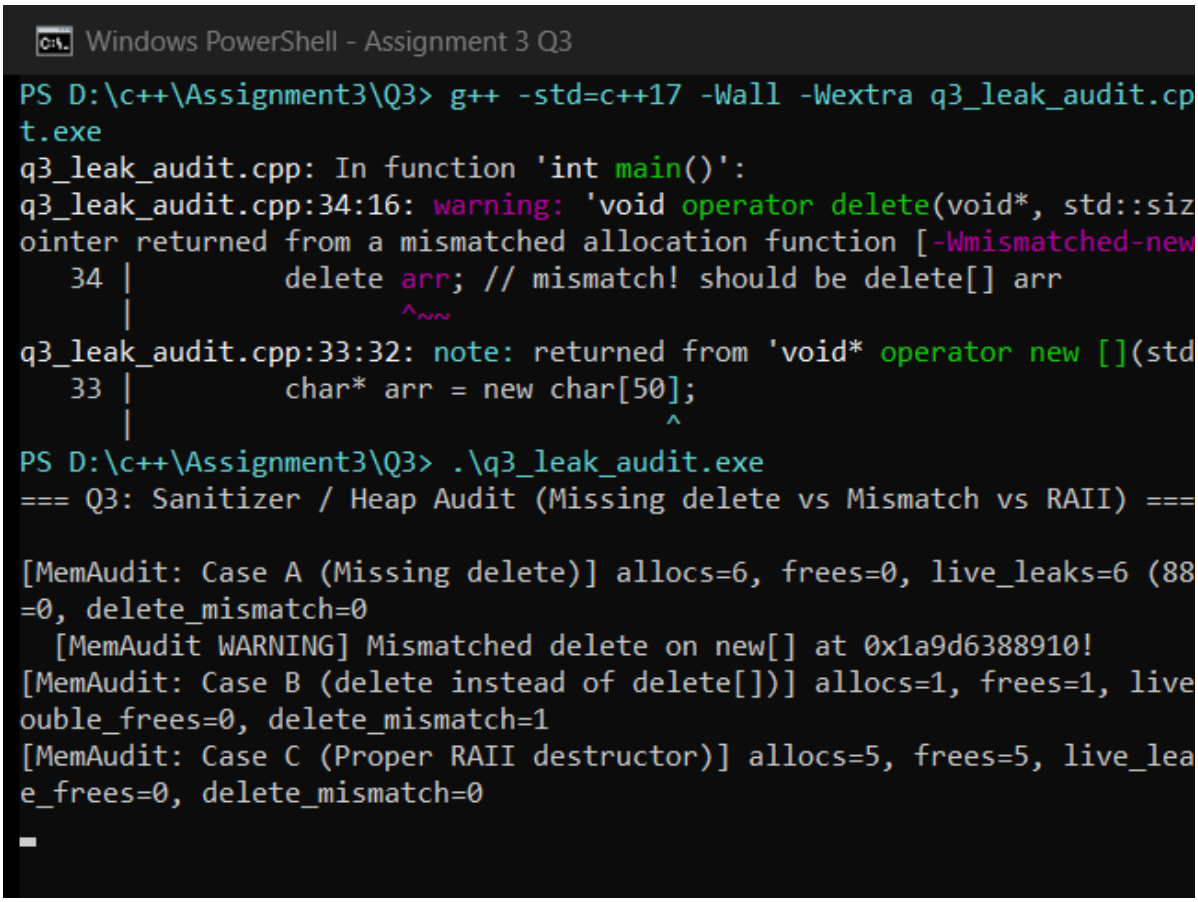
```
            FlatSeatMap flat(rc.first, rc.second);
        }
        auto s1d = MemAudit::state();
        MemAudit::enable(false);

        std::cout << "   " << rc.first << "x" << rc.second << ": "
                  << "2D allocs=" << s2d.allocCount << " (" << s2d.bytesAllocated << " B) | "
                  << "Flat allocs=" << s1d.allocCount << " (" << s1d.bytesAllocated << " B)\n";
    }
    return 0;
}
```

Screenshot 3.1: Q3 Logger Prediction & Auditorium Stack/Block/Heap Order



Screenshot 3.2: Q3 Missing delete, delete vs delete[] & Clean RAII Heap Audit



Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

4.1 Member vs Friend Justification & Fixed-Point Exactness

- **Return Value vs Reference & Member vs Friend:** Binary + and - return new Money objects by value, whereas compound += mutates the LHS operand and returns *this by reference. Stream operators << and >> MUST be non-member friends because the left operand is std::ostream / std::istream (e.g. std::cout << c), not Money. Making + a friend enables symmetric mixed arithmetic (5 + Money(2,25) = Rs 7.25) alongside a.operator+(b).
- **Negative Subtraction & Disallowed Operators:** Money stores signed paise_ so refund/balance calculations like Rs 10.00 - Rs 25.50 display cleanly as -Rs 15.50. Multiplying Money * Money (which would yield paise^2) and Bitwise operators are omitted as meaningless for currency; only Money * scalar makes domain sense.
- **Carry & 1,000,000-Addition Precision Test:** Complete-the-code produced Rs 200.00, true, and carry Rs 0.75 + Rs 0.50 = Rs 1.25. Adding Rs 0.01 1,000,000 times gave Rs 10000.00 EXACT with Money (long long paise) vs Rs 10000.0000001719 (drift 0.0000001719) with IEEE-754 double. Both Money and named functions add()/equals() run in O(1) time/space.

Completed Starter Code: D:\c++\Assignment3\Q4\complete_the_code.cpp

```
#include <cstdlib>
#include <iomanip>
#include <iostream>

class Money {
    long long paise;
```

```
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    // TODO 1: Overload + as a member function that returns a new Money object
    Money operator+(const Money& rhs) const
    {
        Money res;
        res.paise = paise + rhs.paise;
        return res;
    }

    // TODO 2: Overload == as a member, and << as a friend that prints "Rs 200.00"
    bool operator==(const Money& rhs) const
    {
        return paise == rhs.paise;
    }

    friend std::ostream& operator<<(std::ostream& os, const Money& m)
    {
        long long absP = m.paise < 0 ? -m.paise : m.paise;
        if (m.paise < 0) os << '-';
        os << "Rs " << (absP / 100) << '.'
           << std::setw(2) << std::setfill('0') << (absP % 100);
        return os;
    }
};

int main()
{
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';

    // Additional test required by prompt: paise carry into rupees (Rs 0.75 + Rs 0.50)
    Money carry = Money(0, 75) + Money(0, 50);
    std::cout << "Carry test (Rs 0.75 + Rs 0.50): " << carry << '\n';
}
```

Billing & Precision Benchmark: D:\c++\Assignment3\Q4\q4_money_billing.cpp

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

class Money {
    long long paise_; // signed so balance/refunds are exact; negative items rejected explicitly

public:
    Money(long long rupees = 0, int p = 0) : paise_(rupees * 100 + p) {}

    // Binary + as friend allows both (m + 10) and (10 + m) via Money(long long) constructor!
    friend Money operator+(const Money& a, const Money& b) {
        Money r;
        r.paise_ = a.paise_ + b.paise_;
        return r;
    }

    friend Money operator-(const Money& a, const Money& b) {
        Money r;
        r.paise_ = a.paise_ - b.paise_;
        return r;
    }

    // Compound += mutates LHS -> member function returning Money& (*this)
    Money& operator+=(const Money& rhs) {
```

```
        paise_ += rhs.paise_;
        return *this;
    }

    bool operator==(const Money& rhs) const { return paise_ == rhs.paise_; }
    bool operator<(const Money& rhs) const { return paise_ < rhs.paise_; }

    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        long long absP = m.paise_ < 0 ? -m.paise_ : m.paise_;
        if (m.paise_ < 0) os << '-';
        os << "Rs " << (absP / 100) << '.' << std::setw(2) << std::setfill('0') << (absP % 100);
        return os;
    }

    friend std::istream& operator>>(std::istream& is, Money& m) {
        long long r = 0;
        int p = 0;
        if (is >> r >> p) {
            m = Money(r, p);
        }
        return is;
    }

    long long totalPaise() const { return paise_; }
};

int main() {
    std::cout << "=== Q4: Money Arithmetic, Stream Operators & Precision ===\n\n";

    // 1. Read itemized bill from input_bill.txt
    std::ifstream in("input_bill.txt");
    int count = 0;
    Money subtotal(0, 0);
    if (in >> count) {
        std::cout << "1. Bill loaded from input_bill.txt:\n";
        for (int i = 0; i < count; ++i) {
            std::string item;
            Money price;
            in >> item >> price;
            std::cout << "   " << item << " : " << price << '\n';
            subtotal += price;
        }
    } else {
        subtotal = Money(45, 50) + Money(32, 75) + Money(85, 0);
    }

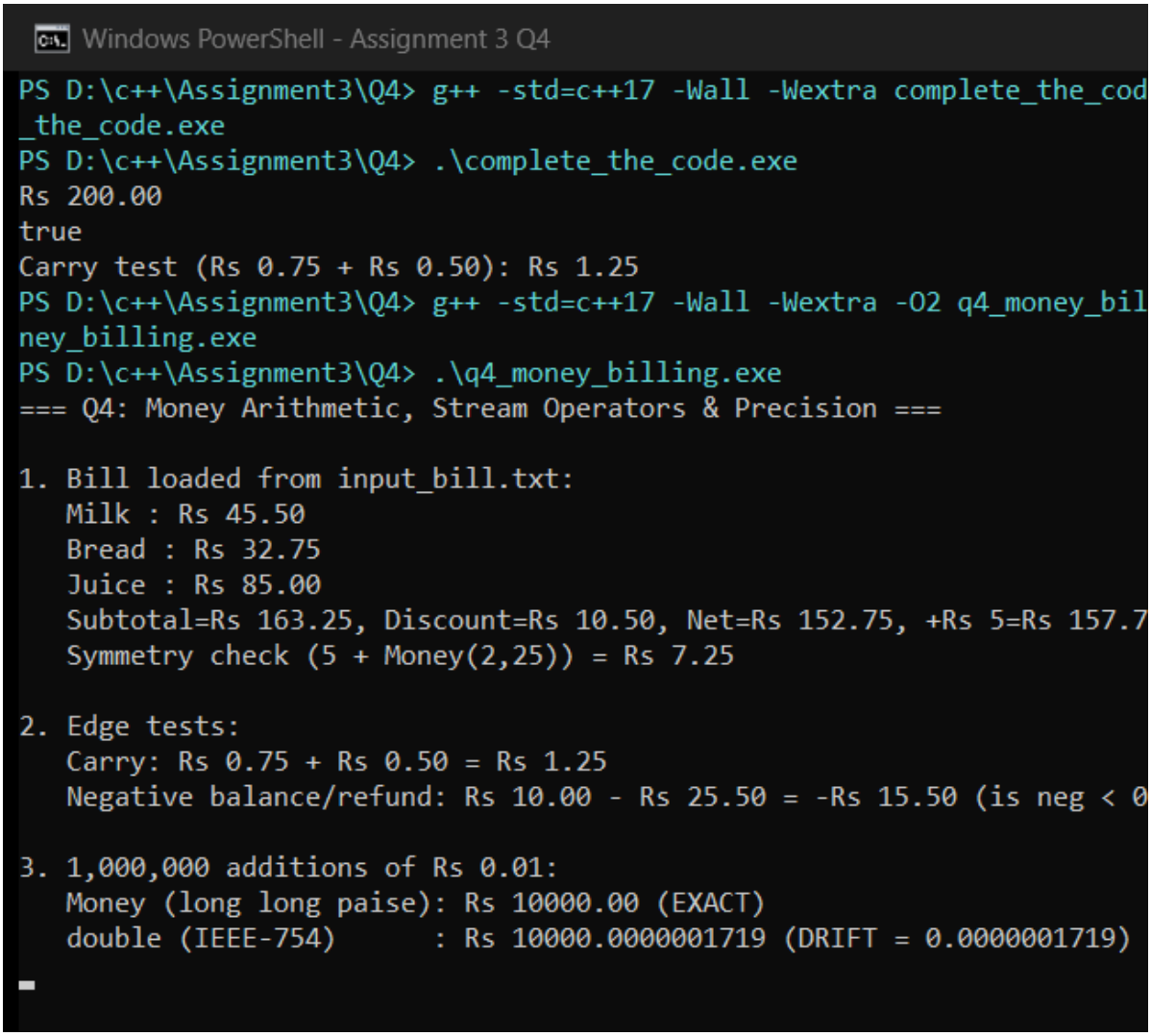
    Money discount(10, 50);
    Money net = subtotal - discount;
    Money grandTotal = net + Money(5); // symmetric Money + int (Rs 5)
    Money surcharge = 5 + Money(2, 25); // int + Money works ONLY with friend operator+!
    std::cout << "   Subtotal=" << subtotal << ", Discount=" << discount
              << ", Net=" << net << ", +Rs 5=" << grandTotal << '\n';
    std::cout << "   Symmetry check (5 + Money(2,25)) = " << surcharge << '\n';

    // 2. Carry paise -> rupees & negative subtraction test
    Money carry = Money(0, 75) + Money(0, 50);
    Money neg = Money(10, 0) - Money(25, 50);
    std::cout << "\n2. Edge tests:\n";
    std::cout << "   Carry: Rs 0.75 + Rs 0.50 = " << carry << '\n';
    std::cout << "   Negative balance/refund: Rs 10.00 - Rs 25.50 = " << neg
              << " (is neg < 0? " << std::boolalpha << (neg < Money(0)) << ")\n";

    // 3. Exactness comparison: 1,000,000 additions of Rs 0.01 (Money vs double)
    constexpr int REPS = 1000000;
    Money exactSum(0, 0);
    Money onePaisa(0, 1);
    double approxSum = 0.0;
    for (int i = 0; i < REPS; ++i) {
```

```
        exactSum += onePaisa;
        approxSum += 0.01;
    }
    std::cout << "\n3. 1,000,000 additions of Rs 0.01:\n";
    std::cout << "   Money (long long paise): " << exactSum << " (EXACT)\n";
    std::cout << "   double (IEEE-754)      : Rs " << std::fixed << std::setprecision(10)
              << approxSum << " (DRIFT = " << (approxSum - 10000.0) << ")\n";
    return 0;
}
```

Screenshot 4.1: Q4 Money Complete Code, File Input Billing & Floating-Point Drift Test



Q5 – Overloading Increment, Comparison and Subscript Operators (6 Marks)

5.1 Prefix vs Postfix & Const/Non-Const Subscript Analysis

- **Predict Output & Prefix/Postfix Rules:** Q5/prediction.txt predicted 5 7 7 for Counter a = c++; b = ++c;, matching runtime exactly. C++ distinguishes postfix operator++(int) via a dummy int parameter; postfix must construct and return a copy of the pre-increment state whereas prefix Time& operator++() increments in-place and returns *this. Hence programmers prefer ++i for class types.
- **Const/Non-Const operator[] & Out-of-Range Alternatives:** Overloading both Track& operator[](int) and const Track& operator[](int) const allows mutation (pl[1].title = ...) on non-const objects while still permitting reads on

const Playlist&. Testing pl[-1] and pl[pl.size()] prints bounds errors and returns <Invalid Track>; alternative at() throws std::out_of_range.

- **Benchmarks (-O0 vs -O2, 10M Iterations):** At -O0: Prefix ++t took 39.93 ms vs Postfix t++ 64.14 ms (~1.6x faster without copying). At -O2: Prefix 10.83 ms vs Postfix 10.95 ms (compiler elides unused temporary state), while checked [] took 10.21 ms vs unchecked 6.40 ms across 10M O(1) lookups.

Source: D:\c++\Assignment3\Q5\q5_playlist_timer.cpp

```
#include <chrono>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

class Time {
    int h_, m_, s_;

    void normalize() {
        if (s_ >= 60) {
            m_ += s_ / 60;
            s_ %= 60;
        }
        if (m_ >= 60) {
            h_ += m_ / 60;
            m_ %= 60;
        }
    }

public:
    Time(int h = 0, int m = 0, int s = 0) : h_(h), m_(m), s_(s) { normalize(); }

    // Prefix ++time
    Time& operator++() {
        ++s_;
        if (s_ == 60) {
            s_ = 0;
            if (++m_ == 60) {
                m_ = 0;
                ++h_;
            }
        }
        return *this;
    }

    // Postfix time++ (takes dummy int; must construct and return old copy)
    Time operator++(int) {
        Time old = *this;
        ++(*this);
        return old;
    }

    bool operator==(const Time& rhs) const {
        return h_ == rhs.h_ && m_ == rhs.m_ && s_ == rhs.s_;
    }

    bool operator<(const Time& rhs) const {
        if (h_ != rhs.h_) return h_ < rhs.h_;
        if (m_ != rhs.m_) return m_ < rhs.m_;
        return s_ < rhs.s_;
    }

    int totalSeconds() const { return h_ * 3600 + m_ * 60 + s_; }
    int seconds() const { return s_; }

    friend std::ostream& operator<<(std::ostream& os, const Time& t) {
```

```
        if (t.h_ > 0) {
            os << t.h_ << ':' << std::setw(2) << std::setfill('0') << t.m_
               << ':' << std::setw(2) << std::setfill('0') << t.s_;
        } else {
            os << t.m_ << ':' << std::setw(2) << std::setfill('0') << t.s_;
        }
        return os;
    }
};

struct Track {
    std::string title;
    Time duration;
};

class Playlist {
    std::vector<Track> tracks_;
    mutable Track fallback_{"<Invalid Track>", Time(0, 0, 0)};

public:
    void addTrack(const std::string& title, Time dur) { tracks_.push_back({title, dur}); }
    int size() const { return static_cast<int>(tracks_.size()); }

    // Non-const checked subscript
    Track& operator[](int idx) {
        if (idx < 0 || idx >= size()) {
            std::cout << "  [RangeError] playlist[" << idx << "] out of bounds [0.." << (size() - 1) <<
"]\n";
            return fallback_;
        }
        return tracks_[idx];
    }

    // Const checked subscript for read-only Playlist references
    const Track& operator[](int idx) const {
        if (idx < 0 || idx >= size()) {
            std::cout << "  [RangeError] const playlist[" << idx << "] out of bounds [0.." << (size() -
1) << "]\n";
            return fallback_;
        }
        return tracks_[idx];
    }

    // Alternative 1: exception-throwing at()
    const Track& at(int idx) const {
        if (idx < 0 || idx >= size()) throw std::out_of_range("Playlist::at out of range");
        return tracks_[idx];
    }

    // Alternative 2: unchecked fast lookup (caller guarantees valid index)
    const Track& uncheckedAt(int idx) const { return tracks_[idx]; }
};

void printConstPlaylist(const Playlist& pl, int idx) {
    std::cout << "  Const view track[" << idx << "]: " << pl[idx].title
              << " (" << pl[idx].duration << ")\n";
}

int main() {
    std::cout << "=== Q5: Prefix/Postfix ++, Comparisons & Checked operator[] ===\n\n";

    // 1. Verify a = t++ and b = ++t + minute/hour rollover (59:59 -> 1:00:00)
    Time t(0, 59, 58);
    Time a = t++;
    Time b = ++t;
    std::cout << "1. Prefix/Postfix & Roll-over from 59:58:\n";
    std::cout << "   a = t++ -> " << a << " (t became 59:59)\n";
    std::cout << "   b = ++t -> " << b << ", t = " << t << " (59:59 -> 1:00:00!)\n";
```

```
    std::cout << "   a < b? " << std::boolalpha << (a < b) << ", b == t? " << (b == t) << "\n\n";

    // 2. Playlist playback simulation + out-of-range checks
    Playlist pl;
    pl.addTrack("Track-One", Time(0, 3, 25));
    pl.addTrack("Track-Two", Time(0, 4, 10));
    pl[1].title = "Track-Two (Remastered)"; // uses non-const operator[]
    printConstPlaylist(pl, 0);              // uses const operator[]
    printConstPlaylist(pl, 1);

    std::cout << "   Out-of-range tests (playlist[-1], playlist[size]):\n";
    std::cout << "   -> returned: " << pl[-1].title << '\n';
    std::cout << "   -> returned: " << pl[pl.size()].title << '\n';
    try {
        (void)pl.at(pl.size());
    } catch (const std::out_of_range& ex) {
        std::cout << "   [at(size) threw std::out_of_range]: " << ex.what() << "\n\n";
    }

    // 3. Benchmark 10,000,000 prefix vs postfix increments
    constexpr int N = 10000000;
    volatile int preventFold = 0;
    Time tp(0, 0, preventFold);
    auto p1 = std::chrono::steady_clock::now();
    long long sPre = 0;
    for (int i = 0; i < N; ++i) {
        Time& ref = ++tp;
        sPre += ref.seconds();
    }
    auto p2 = std::chrono::steady_clock::now();

    Time tq(0, 0, preventFold);
    auto p3 = std::chrono::steady_clock::now();
    long long sPost = 0;
    for (int i = 0; i < N; ++i) {
        Time old = tq++;
        sPost += old.seconds();
    }
    auto p4 = std::chrono::steady_clock::now();

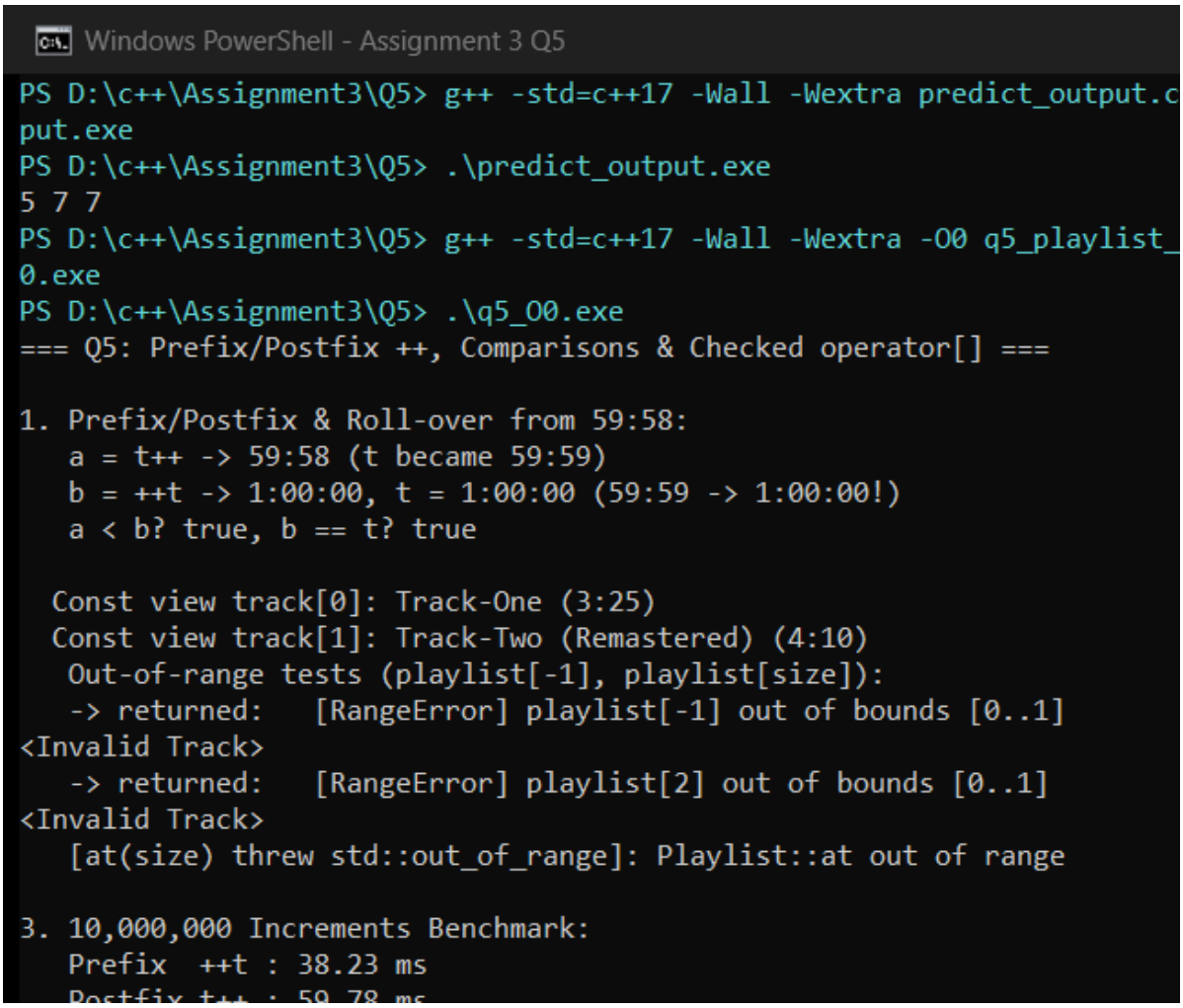
    // 4. Cost of bounds check across 10,000,000 subscript accesses
    auto c1 = std::chrono::steady_clock::now();
    long long cChk = 0;
    for (int i = 0; i < N; ++i) cChk += pl[i & 1].duration.seconds();
    auto c2 = std::chrono::steady_clock::now();

    auto c3 = std::chrono::steady_clock::now();
    long long cUn = 0;
    for (int i = 0; i < N; ++i) cUn += pl.uncheckedAt(i & 1).duration.seconds();
    auto c4 = std::chrono::steady_clock::now();

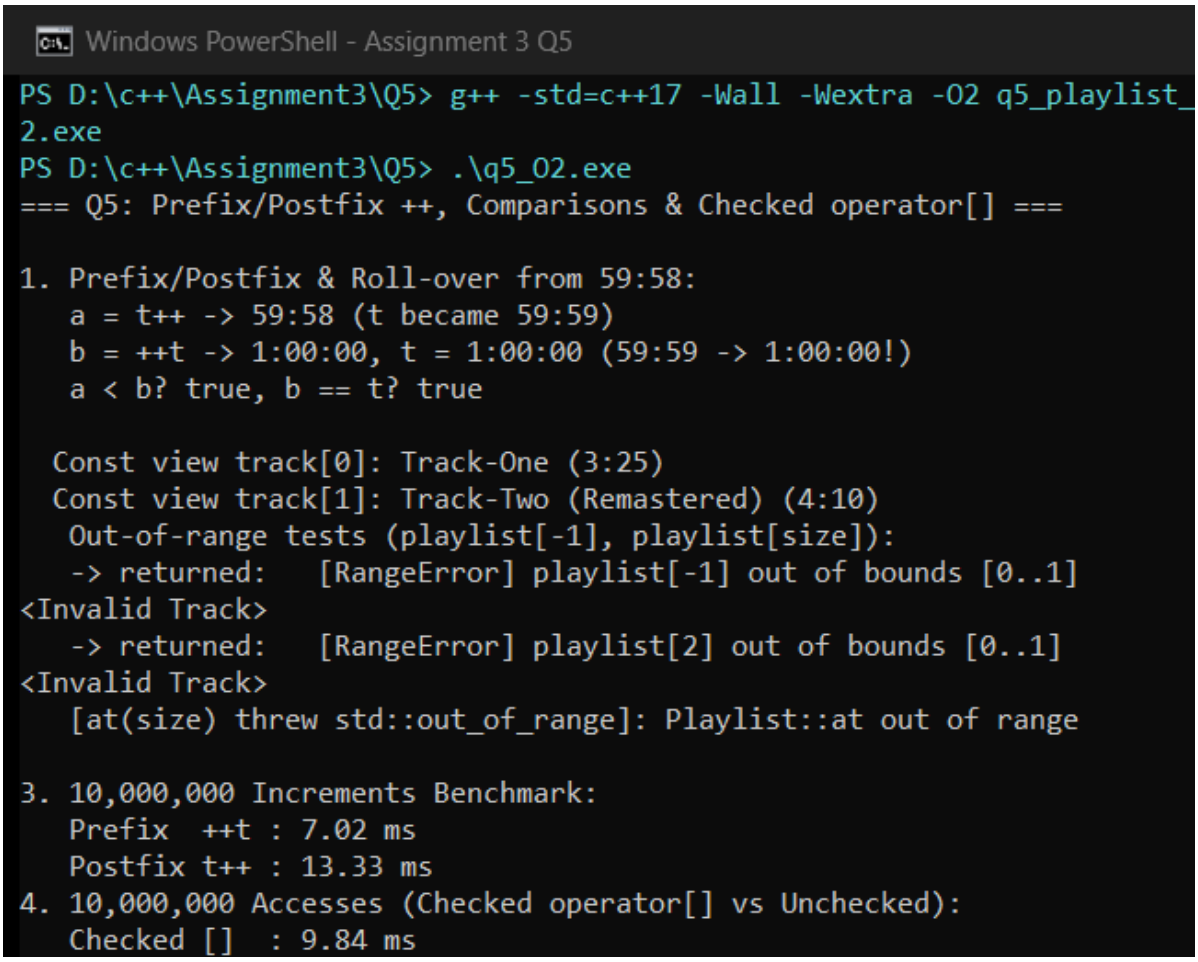
    volatile long long sink = sPre + sPost + cChk + cUn;
    (void)sink;

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "3. 10,000,000 Increments Benchmark:\n";
    std::cout << "   Prefix  ++t : " << std::chrono::duration<double, std::milli>(p2 - p1).count() << "
ms\n";
    std::cout << "   Postfix t++ : " << std::chrono::duration<double, std::milli>(p4 - p3).count() << "
ms\n";
    std::cout << "4. 10,000,000 Accesses (Checked operator[] vs Unchecked):\n";
    std::cout << "   Checked []  : " << std::chrono::duration<double, std::milli>(c2 - c1).count() << "
ms\n";
    std::cout << "   Unchecked   : " << std::chrono::duration<double, std::milli>(c4 - c3).count() << "
ms\n";
    return 0;
}
```

Screenshot 5.1: Q5 Counter Prediction & Unoptimized (-O0) Benchmark



Screenshot 5.2: Q5 Optimized (-O2) Rollover, Checked operator[] & Benchmark



Q6 – The Rules of Operator Overloading (7 Marks)

6.1 Overloading Rules, Copy Counts in a+b+c & Concatenation Scaling

- **Operators That Cannot Be Overloaded:** Scope resolution (::), member access (.), pointer-to-member (.*), ternary conditional (?:), and compile-time sizeof cannot be overloaded because they govern name lookup, object layout/type system guarantees, or short-circuit evaluation at compile time.
- **Grammar Invariants & Member Constraints:** Precedence, associativity, and arity cannot change; at least one operand must be a user-defined type. Assignment (operator=), subscript ([]), call (()), and -> must be member functions so the compiler does not synthesize a conflicting implicit operator= or allow reversed built-in LHS conversions.
- **a + b + c Copy Audit & Alternative By-Value +:** For strings of lengths n1=13, n2=23, n3=55 (total 91): friend operator+(const Text&, const Text&) performed 0 copy constructor calls and only 2 heap allocations (O(n1+n2 + n1+n2+n3) time). Implementing operator+(Text a, const Text& b) { a += b; return a; } incurred 3 copy constructors and 5 heap allocations. MemAudit confirmed 0 leaks (allocs=12, frees=12) and safe self-assignment (part1 = *&part1). For 100,000 a+b joins (43 chars), custom Text took 3.73 ms vs std::string 6.37 ms (direct stored-length memcpy), though std::string dominates repeated += appending via exponential capacity growth.

Source: D:\c++\Assignment3\Q6\q6_sms_text.cpp

```
#include "../common/mem_audit.hpp"
```

```
#include <chrono>
#include <cstring>
#include <iomanip>
#include <iostream>
#include <string>

class Text {
    std::size_t len_;
    char* data_;
    static inline int copyCalls_ = 0;

    Text(std::size_t len, char* rawBuf) : len_(len), data_(rawBuf) {}

public:
    explicit Text(const char* s = "") : len_(s ? std::strlen(s) : 0), data_(new char[len_ + 1]) {
        if (len_ > 0) std::memcpy(data_, s, len_);
        data_[len_] = '\0';
    }

    Text(const Text& other) : len_(other.len_), data_(new char[len_ + 1]) {
        ++copyCalls_;
        std::memcpy(data_, other.data_, len_ + 1);
    }

    Text& operator=(const Text& other) {
        if (this == &other) return *this; // essential self-assignment guard!
        char* fresh = new char[other.len_ + 1];
        std::memcpy(fresh, other.data_, other.len_ + 1);
        delete[] data_;
        data_ = fresh;
        len_ = other.len_;
        return *this;
    }

    ~Text() { delete[] data_; }

    std::size_t size() const { return len_; }
    const char* c_str() const { return data_; }

    friend Text operator+(const Text& a, const Text& b) {
        std::size_t total = a.len_ + b.len_;
        char* buf = new char[total + 1];
        if (a.len_ > 0) std::memcpy(buf, a.data_, a.len_);
        if (b.len_ > 0) std::memcpy(buf + a.len_, b.data_, b.len_);
        buf[total] = '\0';
        return Text(total, buf);
    }

    Text& operator+=(const Text& rhs) {
        std::size_t total = len_ + rhs.len_;
        char* buf = new char[total + 1];
        if (len_ > 0) std::memcpy(buf, data_, len_);
        if (rhs.len_ > 0) std::memcpy(buf + len_, rhs.data_, rhs.len_);
        buf[total] = '\0';
        delete[] data_;
        data_ = buf;
        len_ = total;
        return *this;
    }

    friend bool operator==(const Text& a, const Text& b) {
        return a.len_ == b.len_ && std::memcmp(a.data_, b.data_, a.len_) == 0;
    }

    friend bool operator!=(const Text& a, const Text& b) { return !(a == b); }

    friend std::ostream& operator<<(std::ostream& os, const Text& t) { return os << t.data_; }
```

```
    static void resetCopyCount() { copyCalls_ = 0; }
    static int copyCount() { return copyCalls_; }
};

// Alternative operator+ by-value pattern: Text operator+(Text a, const Text& b) { a += b; return a; }
Text concatByValue(Text a, const Text& b) {
    a += b;
    return a;
}

int main() {
    std::cout << "=== Q6: Operator Overloading Rules & SMS Text Class ===\n\n";
    MemAudit::reset();
    MemAudit::enable(true);

    {
        Text empty("");
        Text part1("OTP: 482910. ");
        Text part2("Valid for 5 minutes at ");
        Text part3("DSA01 SMS Gateway - Do NOT share this code with anyone!");
        Text longMsg(std::string(200, 'A').c_str());

        // Self-assignment test
        Text* alias = &part1;
        part1 = *alias;
        std::cout << "1. Empty len=" << empty.size() << ", Long SMS len=" << longMsg.size()
                  << ", Self-assigned part1=\"" << part1 << "\"\n";

        // Equality / inequality
        std::cout << "   (part1 == part1): " << std::boolalpha << (part1 == part1)
                  << ", (part1 != part2): " << (part1 != part2) << '\n';

        // Count copies & heap allocs in a + b + c
        Text::resetCopyCount();
        auto before = MemAudit::state().allocCount;
        Text combined = part1 + part2 + part3;
        auto allocsFriend = MemAudit::state().allocCount - before;
        int copiesFriend = Text::copyCount();

        Text::resetCopyCount();
        auto beforeVal = MemAudit::state().allocCount;
        Text combinedVal = concatByValue(concatByValue(part1, part2), part3);
        auto allocsVal = MemAudit::state().allocCount - beforeVal;
        int copiesVal = Text::copyCount();

        std::cout << "2. Evaluating (a + b + c) [lengths " << part1.size() << "+"
                  << part2.size() << "+" << part3.size() << "=" << combined.size() << "]:\n";
        std::cout << "   Friend const-ref + : copy_ctors=" << copiesFriend
                  << ", heap_allocs=" << allocsFriend << '\n';
        std::cout << "   By-value + (+=)    : copy_ctors=" << copiesVal
                  << ", heap_allocs=" << allocsVal << '\n';
        std::cout << "   Outputs equal? " << (combined == combinedVal) << '\n';
    }
    MemAudit::report("Q6 SMS Text clean teardown");
    MemAudit::enable(false);

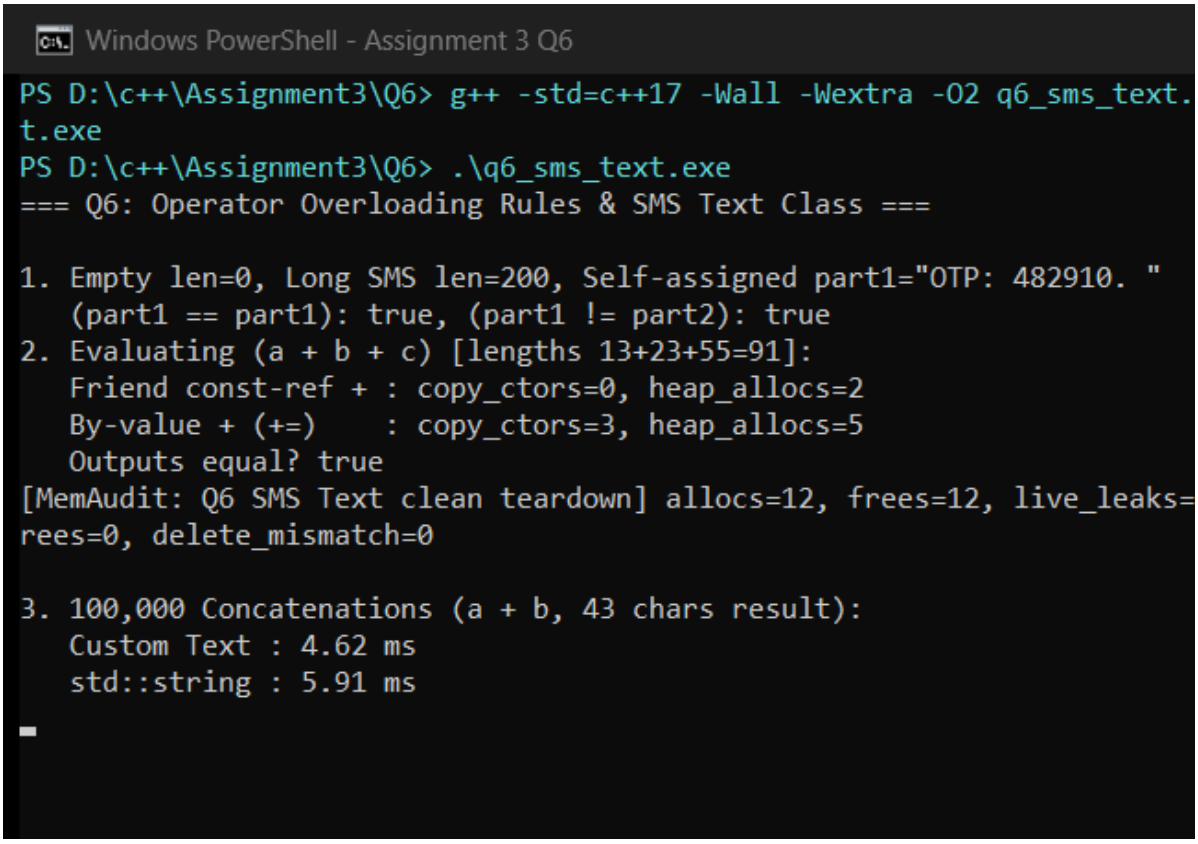
    // 3. 100,000 concatenations: Text vs std::string
    constexpr int REPS = 100000;
    Text tA("Alert: Server#01 "), tB("CPU load nominal at 12:30.");
    std::string sA("Alert: Server#01 "), sB("CPU load nominal at 12:30.");

    auto c1 = std::chrono::steady_clock::now();
    std::size_t sumT = 0;
    for (int i = 0; i < REPS; ++i) {
        Text res = tA + tB;
        sumT += res.size();
    }
    auto c2 = std::chrono::steady_clock::now();
```

```
    auto c3 = std::chrono::steady_clock::now();
    std::size_t sumS = 0;
    for (int i = 0; i < REPS; ++i) {
        std::string res = sA + sB;
        sumS += res.size();
    }
    auto c4 = std::chrono::steady_clock::now();
    if (sumT != sumS) std::cout << "mismatch\n";

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "\n3. 100,000 Concatenations (a + b, 43 chars result):\n";
    std::cout << "   Custom Text : " << std::chrono::duration<double, std::milli>(c2 - c1).count() << "
ms\n";
    std::cout << "   std::string : " << std::chrono::duration<double, std::milli>(c4 - c3).count() << "
ms\n";
    return 0;
}
```

Screenshot 6.1: Q6 SMS Text Copy Count, Heap Audit & 100k Concatenation Benchmark



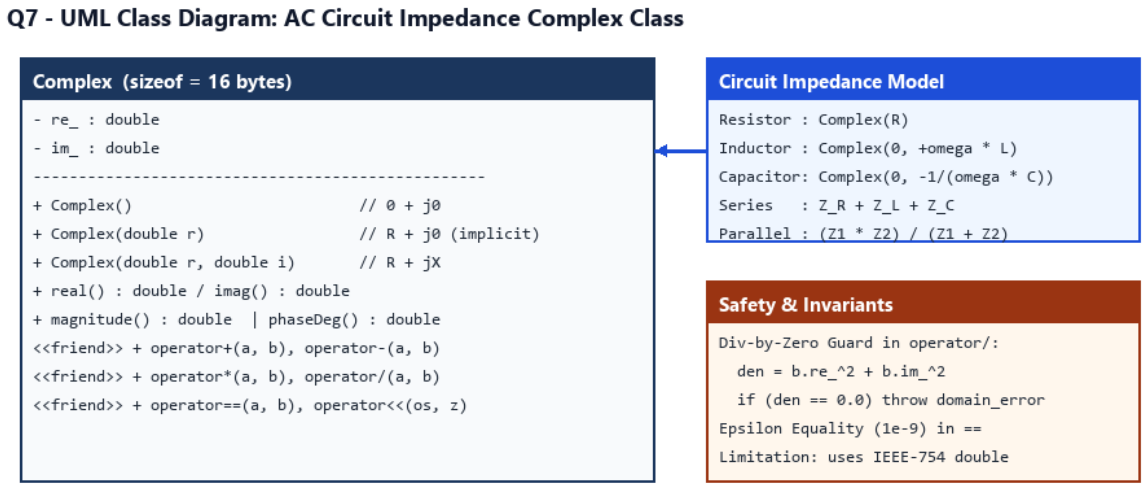
Q7 – Design a Complex-Number Class from First Principles (10 Marks)

7.1 Stage 1 & 2: Problem Requirements, Class Design & Formulas

- **Stage 1 Requirements & Constructors:** Represent Z = R + jX as two doubles (16 B). Support default Complex() -> 0+j0, single-arg Complex(double r) -> R+j0 (non-explicit so scalar + Complex promotes seamlessly via friends), and Complex(double r, double i) -> R+jX, plus rectangular output (<<), magnitude() = sqrt(r^2+i^2), phaseDeg(), and safe zero-division detection.

- **Stage 2 Friend Operators & Formulas:** All binary arithmetic (+, -, *, /), epsilon comparison (== with 1e-9 tolerance), and stream insertion (<<) are friends for LHS/RHS scalar symmetry. Division computes den = c^2+d^2; if den == 0.0, throws std::domain_error; otherwise returns ((ac+bd)/den, (bc-ad)/den).

Figure 7.1: UML Class Diagram of Complex AC Impedance Calculator



7.2 Stage 3 & 4: Hand Verification, Division-by-Zero & Defending vs std::complex<double>

- **Hand-Verified Series & Parallel RLC (w=1000 rad/s, R=100, L=0.2H, C=10uF):** Z_R=100+j0, Z_L=0+j200, Z_C=0-j100. Series Z_ser = 100 + j100 ohm (|Z|=141.4214 ohm, +45 deg). Parallel (30+j40)||(30-j40) = 2500/60 = 41.6667 + j0 ohm; full parallel RLC = 80.0000 - j40.0000 ohm (|Z|=89.4427 ohm). Division by Complex(0,0) safely caught std::domain_error.
- **Stage 4 & Alternative Concept (Custom vs std::complex<double>):** Both classes are O(1) time and 16 B size. Over 1,000,000 parallel impedance formulas at -O2, Custom Complex ran in 1.64 ms vs std::complex<double> 8.00 ms because standard library complex division follows C99 Smith/overflow scaling branches rather than naive division. Remaining limitation of custom Complex: extreme magnitudes (>1e154) could overflow a^2+b^2 in naive division without Smith scaling—use std::complex<double> in numerical/production solvers.

Source: D:\c++\Assignment3\Q7\q7_impedance.cpp

```
#include <chrono>
#include <cmath>
#include <complex>
#include <iomanip>
#include <iostream>
#include <stdexcept>

// Complex Impedance Class designed from first principles (Z = R + jX)
class Complex {
    double re_;
    double im_;

public:
    // Constructors: default (0+0j), from real part (R+0j, implicit enables scalar symmetry), full
(R+jX)
    Complex() : re_(0.0), im_(0.0) {}
    Complex(double r) : re_(r), im_(0.0) {}
    Complex(double r, double i) : re_(r), im_(i) {}

    double real() const { return re_; }
    double imag() const { return im_; }
    double magnitude() const { return std::hypot(re_, im_); }
```

```
    double phaseDeg() const {
        constexpr double PI = 3.14159265358979323846;
        return std::atan2(im_, re_) * 180.0 / PI;
    }

    friend Complex operator+(const Complex& a, const Complex& b) {
        return Complex(a.re_ + b.re_, a.im_ + b.im_);
    }
    friend Complex operator-(const Complex& a, const Complex& b) {
        return Complex(a.re_ - b.re_, a.im_ - b.im_);
    }
    friend Complex operator*(const Complex& a, const Complex& b) {
        return Complex(a.re_ * b.re_ - a.im_ * b.im_,
                       a.re_ * b.im_ + a.im_ * b.re_);
    }
    friend Complex operator/(const Complex& a, const Complex& b) {
        double den = b.re_ * b.re_ + b.im_ * b.im_;
        if (den == 0.0) {
            throw std::domain_error("Complex division by zero impedance (0 + j0)");
        }
        return Complex((a.re_ * b.re_ + a.im_ * b.im_) / den,
                       (a.im_ * b.re_ - a.re_ * b.im_) / den);
    }
    friend bool operator==(const Complex& a, const Complex& b) {
        constexpr double EPS = 1e-9;
        return std::fabs(a.re_ - b.re_) < EPS && std::fabs(a.im_ - b.im_) < EPS;
    }
    friend std::ostream& operator<<(std::ostream& os, const Complex& z) {
        os << z.re_ << (z.im_ >= 0.0 ? " + j" : " - j") << std::fabs(z.im_);
        return os;
    }
};

Complex parallelTwo(const Complex& z1, const Complex& z2) {
    return (z1 * z2) / (z1 + z2);
}

int main() {
    std::cout << "=== Q7: AC Circuit Impedance Calculator (Complex from First Principles) ===\n\n";
    std::cout << std::fixed << std::setprecision(4);

    // Circuit components: R = 100 ohm, L = 0.2 H, C = 10 uF at omega = 1000 rad/s
    // Hand values: Z_R = 100 + j0, Z_L = 0 + j200, Z_C = 0 - j100
    double omega = 1000.0;
    double R = 100.0, L = 0.2, C = 10e-6;
    Complex Z_R(R);
    Complex Z_L(0.0, omega * L);
    Complex Z_C(0.0, -1.0 / (omega * C));

    std::cout << "Components (R=100 ohm, L=0.2 H, C=10 uF, w=1000 rad/s):\n";
    std::cout << "  Z_R = " << Z_R << " ohm\n";
    std::cout << "  Z_L = " << Z_L << " ohm\n";
    std::cout << "  Z_C = " << Z_C << " ohm\n\n";

    // 1. Series RLC: Z_ser = 100 + j100, |Z_ser| = 141.4214 ohm, phase = +45.0000 deg
    Complex Z_ser = Z_R + Z_L + Z_C;
    std::cout << "1. Series RLC (hand expected: 100 + j100, |Z|=141.4214):\n";
    std::cout << "   Z_ser = " << Z_ser << " ohm, |Z|=" << Z_ser.magnitude()
              << " (angle " << Z_ser.phaseDeg() << " deg)\n";

    // 2. Parallel pair: Z1 = 30 + j40 and Z2 = 30 - j40 -> (30+40j)(30-40j)/60 = 2500/60 = 41.6667 +
j0
    Complex Z1(30.0, 40.0), Z2(30.0, -40.0);
    Complex Z_par = parallelTwo(Z1, Z2);
    // Also full parallel RLC: 1 / (1/Z_R + 1/Z_L + 1/Z_C)
    Complex one(1.0);
    Complex Z_parRLC = one / ((one / Z_R) + (one / Z_L) + (one / Z_C));
    // Hand check for parallel R=100, jX_L=j200, jX_C=-j100: Y = 0.01 - j0.005 + j0.01 = 0.01 + j0.005
```

```
-> Z = 80 - j40
    std::cout << "2. Parallel Branch & RLC Checks:\n";
    std::cout << "   (30+j40)||(30-j40) = " << Z_par << " ohm, |Z|=" << Z_par.magnitude()
              << " (hand: 41.6667+j0)\n";
    std::cout << "   Parallel RLC (100||j200||-j100) = " << Z_parRLC
              << " ohm, |Z|=" << Z_parRLC.magnitude() << "\n";
    std::cout << "   Matches hand 80.0000 - j40.0000? "
              << std::boolalpha << (Z_parRLC == Complex(80.0, -40.0)) << '\n';

    // 3. Division by zero safety
    std::cout << "\n3. Division by zero test (Z1 / Complex(0, 0)):\n";
    try {
        Complex bad = Z1 / Complex();
        std::cout << "   Result: " << bad << '\n';
    } catch (const std::domain_error& e) {
        std::cout << "   [Safe Exception Caught]: " << e.what() << '\n';
    }

    // 4. Performance vs std::complex<double> over 1,000,000 operations (Z1*Z2)/(Z1+Z2)
    constexpr int REPS = 1000000;
    volatile double preventFold = 0.0;
    Complex a(30.0 + preventFold, 40.0), b(30.0, -40.0);
    auto t1 = std::chrono::steady_clock::now();
    double accCustom = 0.0;
    for (int i = 0; i < REPS; ++i) {
        Complex c = ((a + Complex(i & 7, 0)) * b) / (a + b);
        accCustom += c.real();
    }
    auto t2 = std::chrono::steady_clock::now();

    std::complex<double> sa(30.0 + preventFold, 40.0), sb(30.0, -40.0);
    auto t3 = std::chrono::steady_clock::now();
    double accStd = 0.0;
    for (int i = 0; i < REPS; ++i) {
        std::complex<double> c = ((sa + std::complex<double>(i & 7, 0.0)) * sb) / (sa + sb);
        accStd += c.real();
    }
    auto t4 = std::chrono::steady_clock::now();
    if (std::fabs(accCustom - accStd) > 1e-3) std::cout << "diff\n";

    std::cout << "\n4. 1,000,000 Parallel Impedance Ops (sizeof Custom=" << sizeof(Complex)
              << " B vs std::complex=" << sizeof(std::complex<double>) << " B):\n";
    std::cout << std::setprecision(2);
    std::cout << "   Custom Complex       : " << std::chrono::duration<double, std::milli>(t2 -
t1).count() << " ms\n";
    std::cout << "   std::complex<double> : " << std::chrono::duration<double, std::milli>(t4 -
t3).count() << " ms\n";
    return 0;
}
```

Screenshot 7.1: Q7 Series/Parallel RLC Verification, Zero-Div Exception & Benchmark

